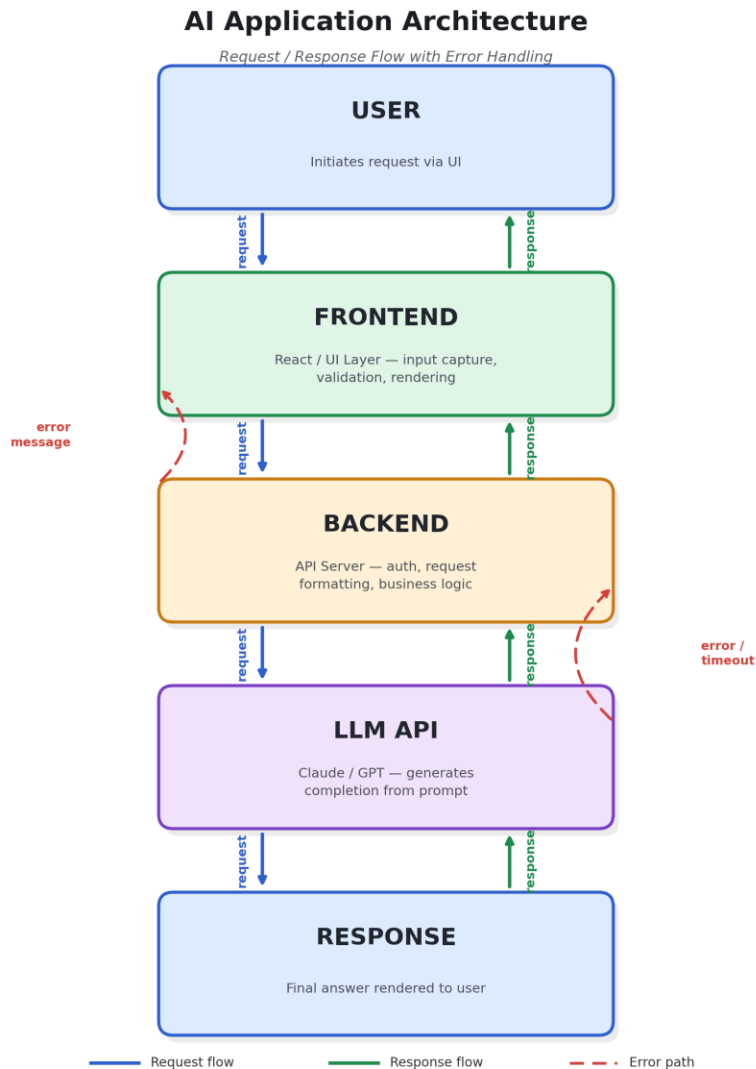


Assignment 5: AI Application Architecture



1. Request Flow

A request begins when the User interacts with the Frontend (e.g., typing a prompt into a chat UI or submitting a form). The Frontend captures the input, performs basic client-side validation, and packages it into a structured request (typically JSON) sent over HTTPS to the Backend API server.

- User Action → the UI event (click/submit) triggers a request handler.
- Frontend → Backend: an HTTP request (e.g., POST /api/chat) carrying the payload and auth token/session.
- Backend → LLM API: the Backend validates the request, attaches system instructions or context, and forwards a properly formatted call to the LLM provider's endpoint using an API key.

2. Response Flow

Once the LLM API processes the prompt, it returns a completion object containing the generated text (and metadata such as token usage). This flows back through the same chain in reverse.

- LLM API → Backend: the raw model response is received and parsed.
- Backend → Frontend: the Backend may post-process the response (formatting, filtering, logging) before sending a clean JSON response back to the client.
- Frontend → User: the UI renders the response, updating the chat window or page in real time (optionally via streaming).

3. Error Handling

Errors can occur at any layer, so each layer implements its own checks with graceful fallback rather than letting failures propagate silently.

- Frontend: input validation, network-failure detection, and user-friendly error banners (e.g., "Message failed to send, retry?").
- Backend: try/catch around the LLM call, handling timeouts, rate limits (HTTP 429), authentication failures (401), and malformed responses; returns standardized error codes to the Frontend.
- LLM API: may return errors for content policy violations, invalid tokens, or context-length overflow — the Backend catches these and maps them to safe, descriptive messages.
- Retry logic: exponential backoff for transient errors (timeouts, 5xx) improves reliability without overwhelming the API.

4. API Integration

The Backend acts as a secure intermediary between the Frontend and the LLM provider, so API keys are never exposed to the client.

- Authentication: the LLM API key is stored server-side (environment variables/secrets manager), never in Frontend code.
- Request formatting: the Backend structures calls per the provider's schema (model name, messages array, max_tokens, temperature, etc.).
- Abstraction layer: a service/module wraps the LLM SDK so the rest of the Backend is decoupled from a specific provider, easing future swaps or multi-model support.
- Monitoring: logging requests, response times, and token usage supports debugging and cost tracking.